Lab 5: TCP Traffic Analysis using Wireshark

EE353 Computer Networks | Department of Computing

Name	Arqam Waheed	**Reg No.**	553514
Class	BSDS 3A	**Date**	8 October 2026

Setup

Wireshark 3.6.2 on Ubuntu, capturing on wlp0s20f3 (192.168.18.55). File: lab5_tcp.pcapng, 1519 packets, 1511 of them TCP. I used curl to load example.com over HTTP and HTTPS and then connected to a closed port on my router. This report mostly follows the HTTPS connection (stream 34).

```
arqamwd@ArqamWd:~$ curl -4 -s -o /dev/null -w "http %{http_code}\n" --http1.1 http://example.com/
http 200
arqamwd@ArqamWd:~$ curl -4 -s -o /dev/null -w "https %{http_code}\n" https://example.com/
https 200
arqamwd@ArqamWd:~$ exec 3<>/dev/tcp/192.168.18.1/81
bash: connect: Connection refused
arqamwd@ArqamWd:~$
```

Figure 1: Commands run during the capture

1. Identify TCP Traffic

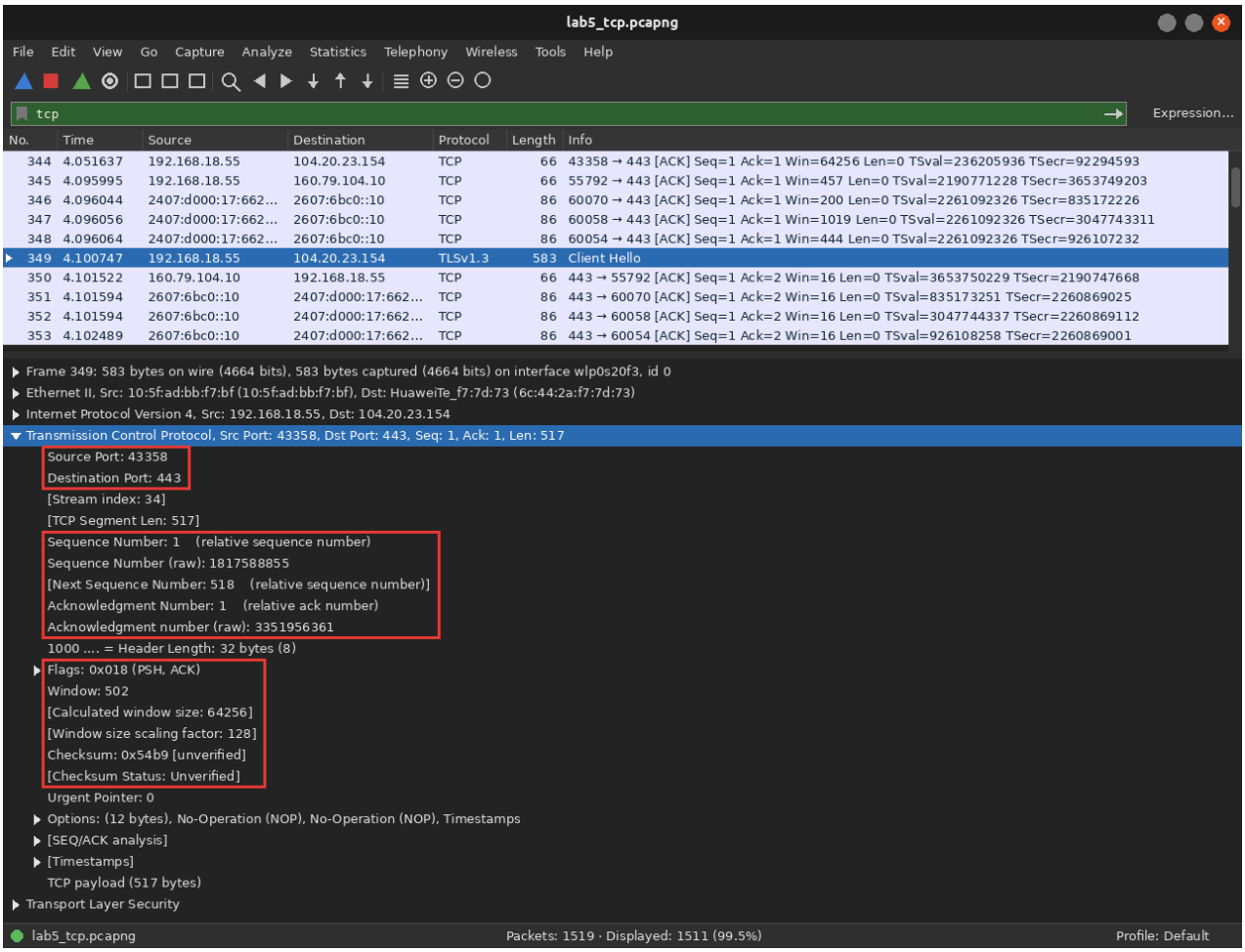


Figure 2: Filter tcp, frame 349 (TLS Client Hello) with the TCP header expanded

Field	Observation
Source IP	192.168.18.55
Destination IP	104.20.23.154
Source Port	43358
Destination Port	443
Sequence Number	1 (raw 1817588855)
Acknowledgment Number	1 (raw 3351956361)
Flags	0x018 (PSH, ACK)
Window Size	502, scaled ×128 = 64256 bytes
TCP Header Length	32 bytes
Checksum / Status	0x54b9, unverified
Higher Layer Protocol	TLS (Client Hello)

Outgoing packets from my laptop show unverified checksums because the Wi-Fi card fills them in after capture (checksum offload).

2. Three-Way Handshake

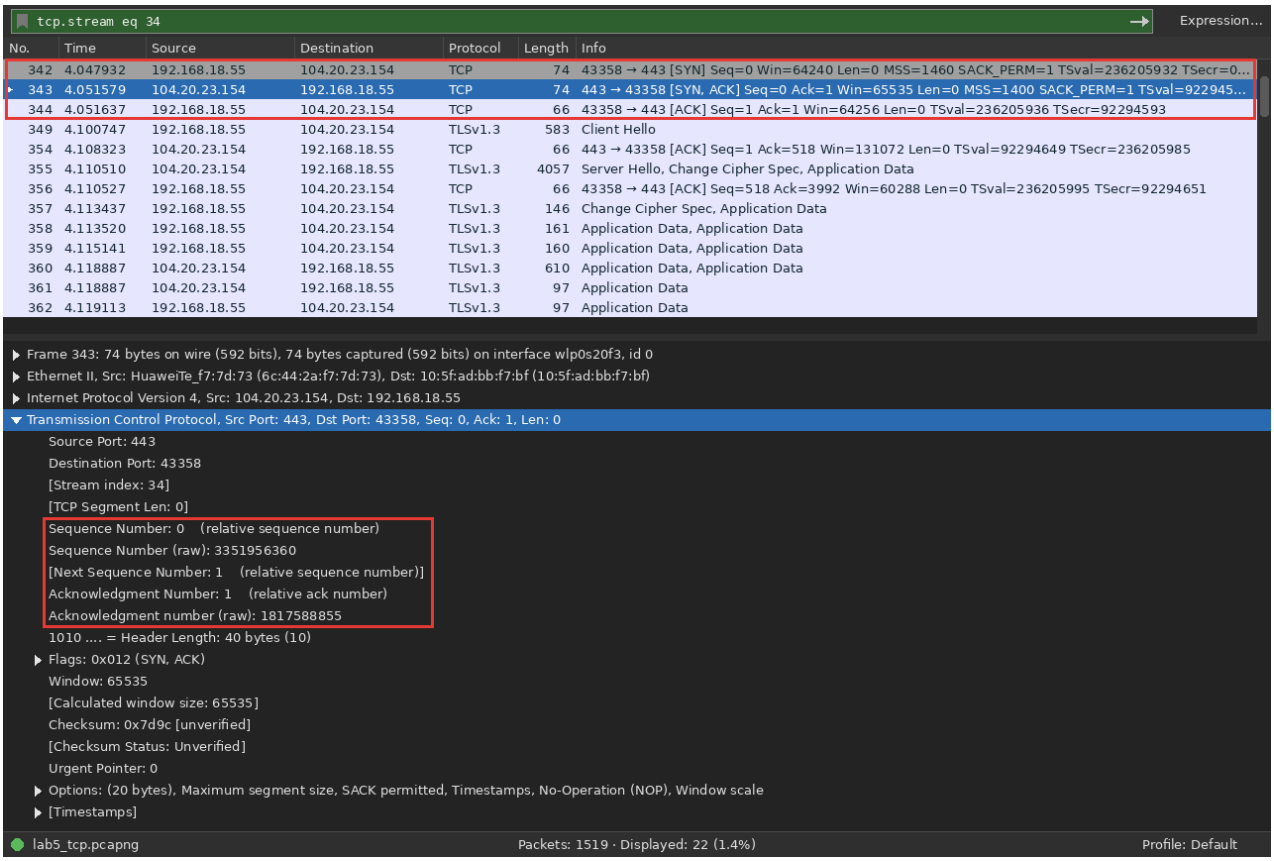


Figure 3: tcp.stream eq 34, handshake in frames 342 to 344

Step	Frame	Ports	Seq (raw)	Ack (raw)
SYN	342	43358 → 443	1817588854	0
SYN/ACK	343	443 → 43358	3351956360	1817588855
ACK	344	43358 → 443	1817588855	3351956361

Q: What is the purpose of SYN?
It requests a new connection and tells the server the client's initial sequence number.

Q: What does SYN/ACK acknowledge?
The client's SYN. Ack 1817588855 is the client's ISN plus one.

Q: Why is the final ACK required?
The server also sent a SYN, so it needs an acknowledgment too. After this both directions are open.

Q: Which endpoint initiated the connection?
My laptop, 192.168.18.55, port 43358.

3. Sequence and Acknowledgment Analysis

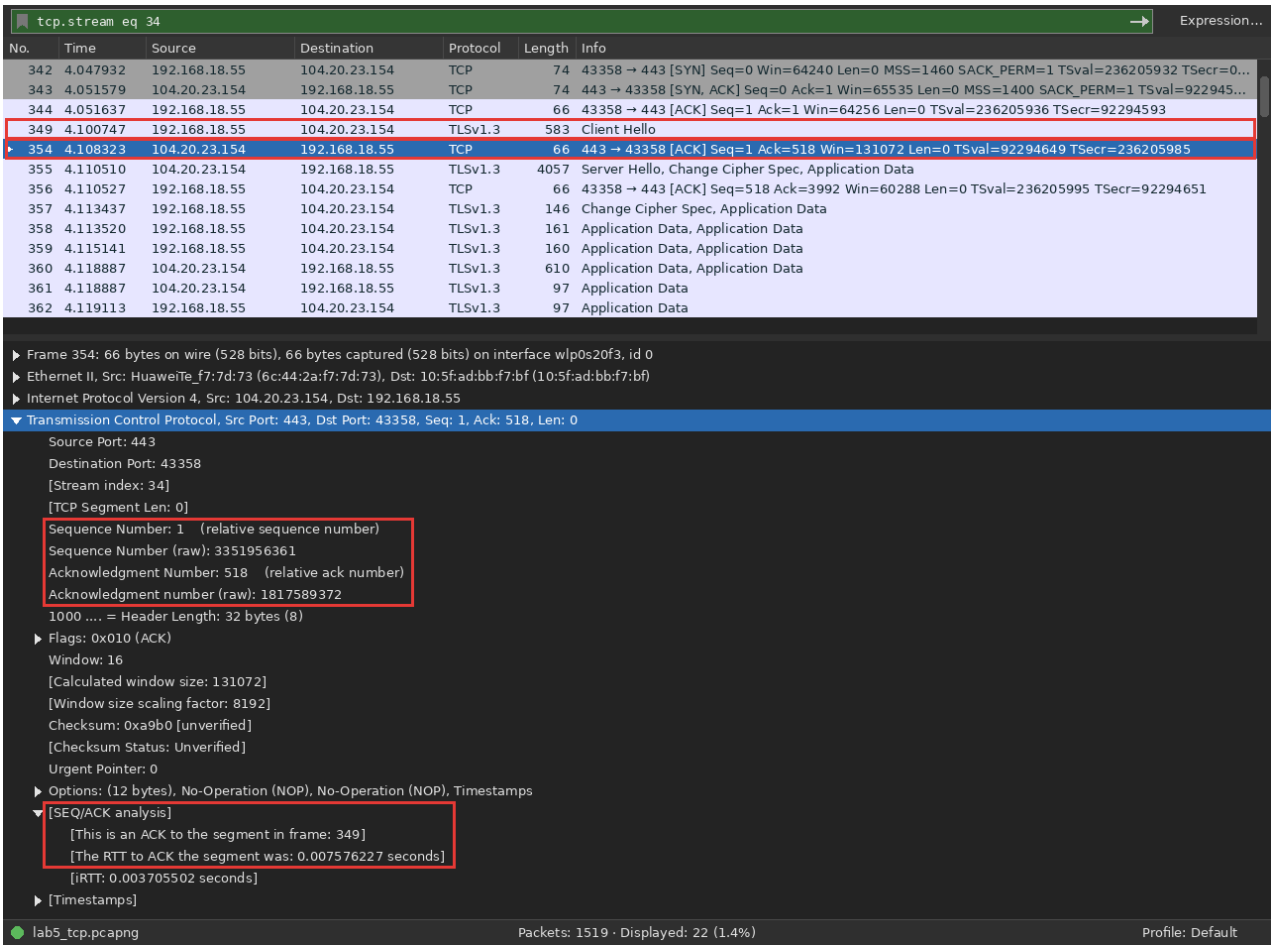


Figure 4: Frame 354 acknowledges the 517-byte Client Hello in frame 349

Item	Value
Data packet	Frame 349, Seq = 1, Len = 517
Next sequence number	518
ACK packet	Frame 354, Ack = 518
Time difference	4.108323 − 4.100747 = 7.6 ms

Q: How do sequence and acknowledgment numbers support reliable, ordered delivery?
Sequence numbers count bytes, not packets. The receiver uses them to reorder segments and find missing ones. The ACK says which byte it wants next: Ack 518 means bytes 1 to 517 arrived. If no ACK comes back the sender retransmits.

4. TCP Flags

Filter	Seen?	Count	Explanation
tcp.flags.syn == 1	Yes	13	SYN and SYN/ACK packets that open connections
tcp.flags.ack == 1	Yes	1501	Ack field is valid; on everything except plain SYNs and RSTs
tcp.flags.fin == 1	Yes	11	Sender is done sending and is closing its side
tcp.flags.reset == 1	Yes	5	Connection refused or aborted

5. Follow TCP Stream

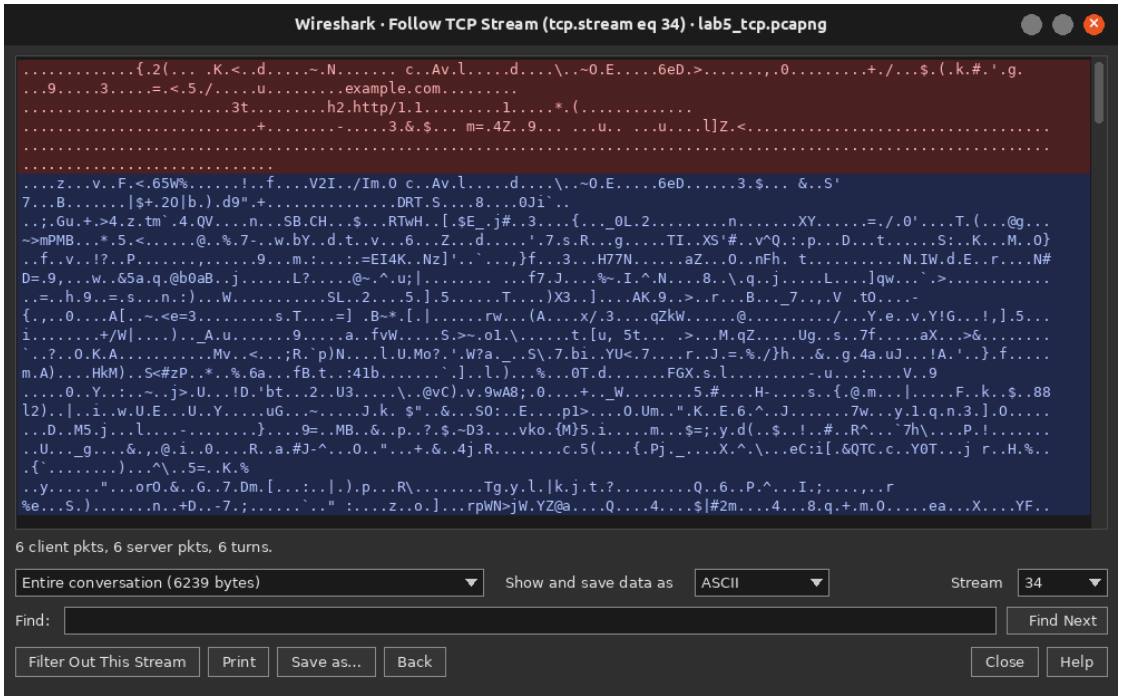


Figure 5: Follow TCP Stream on stream 34 (HTTPS)

Stream 34 is between 192.168.18.55:43358 (client) and 104.20.23.154:443 (server). Wireshark set the filter to tcp.stream eq 34 and showed only its 22 packets (6239 bytes of payload).

Q: What does a TCP stream represent?
One TCP connection: a single pair of IP addresses and ports.

Q: Why is stream-level analysis useful?
A capture has many connections mixed together. One stream shows a single conversation in order.

Q: Why may application data not be readable when HTTPS/TLS is used?
TLS encrypts everything after the handshake. Only example.com in the Client Hello is readable. The HTTP stream (25) showed the full GET and 200 OK in plain text.

6. Connection Termination

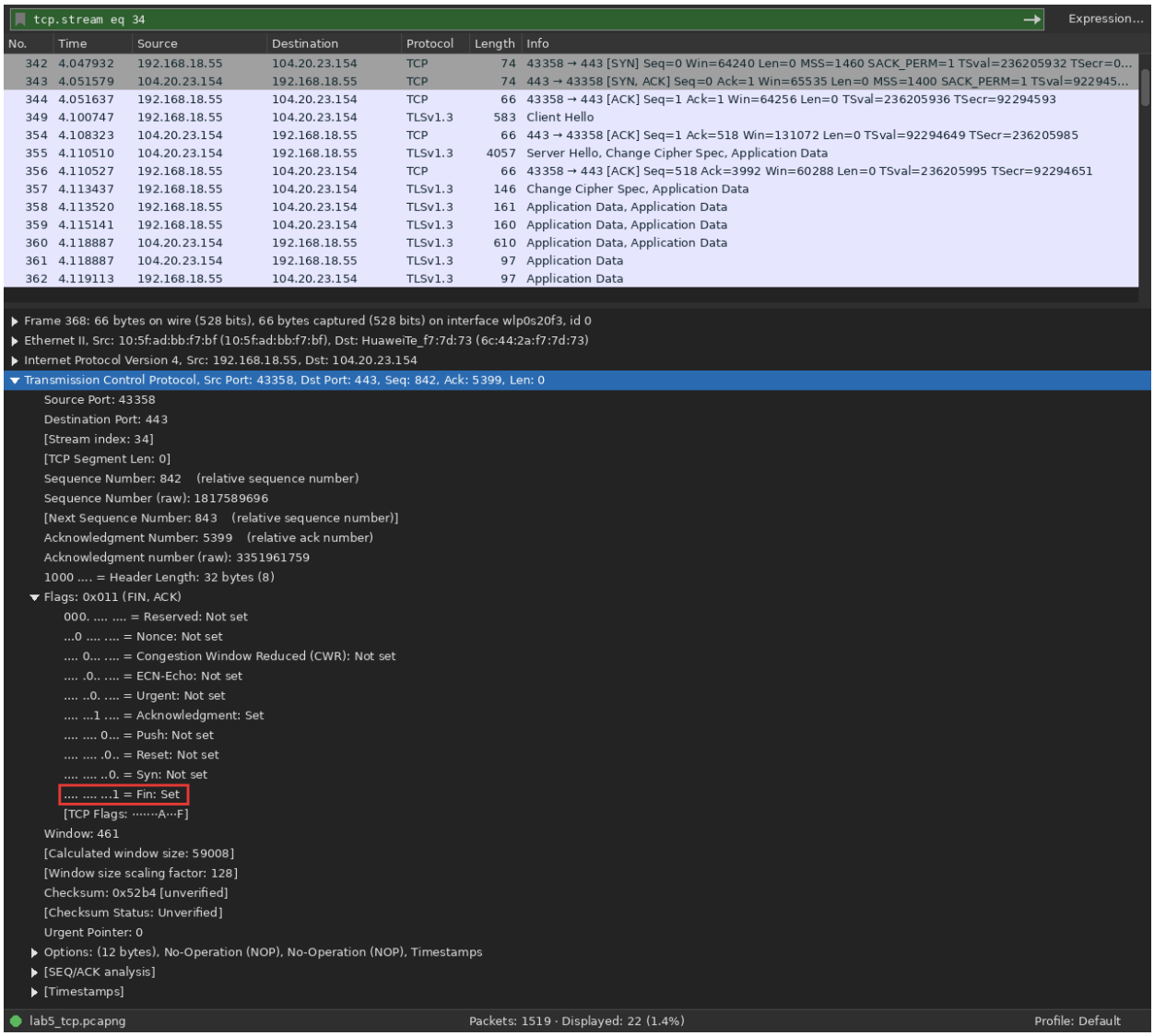


Figure 6: Four-packet close of stream 34 (frames 368 to 371)

Frame	Direction	Flags	Seq	Ack
368	client → server	FIN, ACK	842	5399
369	server → client	ACK	5399	842
370	server → client	FIN, ACK	5399	843
371	client → server	ACK	843	5400

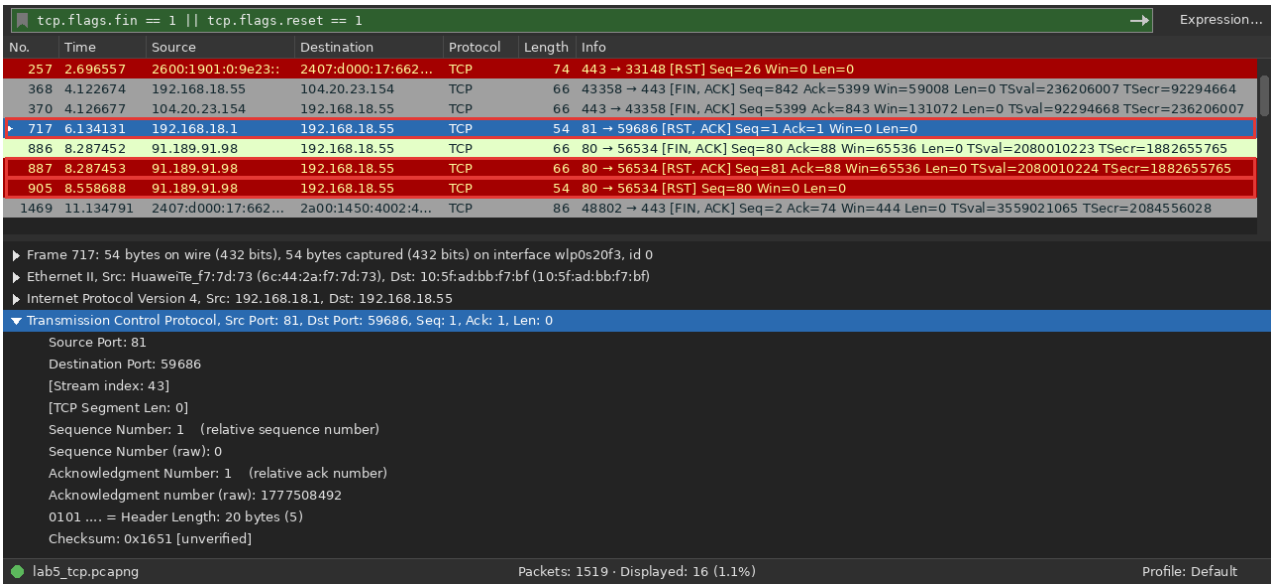


Figure 7: RST packets: router refusing port 81 (717) and Ubuntu's connectivity check (887, 905)

Frame 717 is a RST, ACK from my router because nothing listens on port 81. Frames 887 and 905 are resets from connectivity-check.ubuntu.com, which aborted its connection after sending a FIN.

Q: Why can TCP connection termination involve multiple packets in both directions?
Each direction is closed separately. Both sides send a FIN and both FINs need an ACK, so a close takes 4 packets (3 if one side puts its ACK and FIN together).

7. TCP Port 443

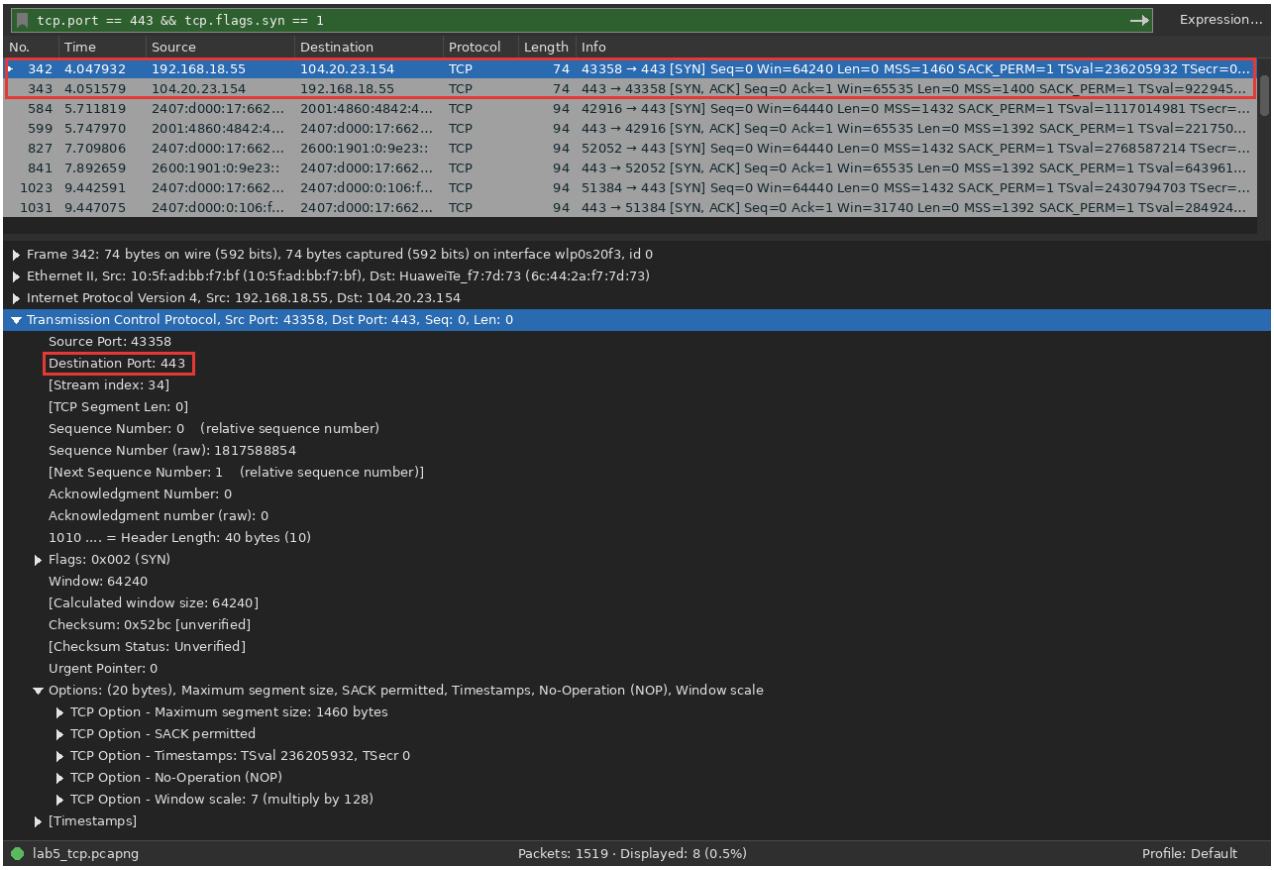


Figure 8: tcp.port == 443 && tcp.flags.syn == 1

TCP/443 traffic was present (1482 packets). The connection to example.com used port 443 and did the SYN (342) → SYN/ACK (343) → ACK (344) handshake before TLS started. Port 443 is usually HTTPS, but any program can use any port. Wireshark only shows it as TLS after reading the payload.

8. TCP vs UDP

Feature	TCP	UDP
Connection setup	3-way handshake	None
Reliability	ACKs + retransmission	None
Ordering	Yes, using sequence numbers	No
Acknowledgments	Yes	No
Flow control	Receive window	No
Minimum transport header	20 bytes	8 bytes
Examples	HTTP, HTTPS, SSH	DNS, DHCP, QUIC

From my captures:

- TCP spent 3 packets (342–344) on the handshake before any data. The Lab 4 DNS lookup was just a query and a response.
- TCP headers here were 32 bytes (with options) vs 8 bytes for every UDP packet in Lab 4.
- Every TCP data segment got an ACK (349 → 354). UDP packets were never acknowledged.

Viva Questions

Q: What is TCP and why is it connection-oriented?
A reliable transport layer protocol. It sets up a connection with a handshake before data is sent.

Q: Explain the three-way handshake.
SYN from client, SYN/ACK from server, ACK from client.

Q: What is the purpose of SYN?
To start a connection and synchronise initial sequence numbers.

Q: Sequence number vs acknowledgment number?
Seq numbers the data being sent. Ack is the next byte expected from the other side.

Q: Why does TCP use acknowledgments?
To confirm delivery so lost data can be retransmitted.

Q: What is the receive window?
Free buffer space at the receiver, which limits how much unacknowledged data the sender can send.

Q: Flow control vs congestion control?
Flow control stops the receiver from being overloaded. Congestion control stops the network from being overloaded.

Q: FIN vs RST?
FIN is a normal, graceful close. RST ends the connection immediately and drops any pending data.

Q: What does Follow TCP Stream do?
Isolates one connection and shows its data as a conversation.

Q: Why might a website not produce TCP/443?
It may use HTTP/3 over QUIC (UDP/443) instead.

Q: What is a TCP segment?
One TCP header plus its payload.

Q: How does TCP differ from UDP?
TCP is reliable, ordered and connection-based. UDP is fast with no guarantees.

Q: What can a retransmission or duplicate ACK indicate?
A lost or out-of-order segment, usually from congestion.